



Norme di progetto

20/11/2025



Revisioni

Ver.	Autore	Modifiche	Data	Verifica
1.2	Stefano Maso	Aggiunte sottosezioni <i>Codifica</i> e <i>Verifica</i>	30/04/2026	Besnik Murtezan
1.1	Niccolò Feltrin	Aggiunta sezione 3.2.5 <i>Scopo e struttura repository "SecondBrain"</i> , aggiornata sezione 3.1.1 (aggiunti documenti Specifica Tecnica e Manuale Utente), aggiornata sezione 2.4.3	01/04/2026	Stefano Maso
1.0	Besnik Murtezan	Approvazione documento per RTB	24/03/2026	Stefano Maso
0.10	Stefano Maso	Aggiunti riferimenti normativi e informativi	24/03/2026	Niccolò Feltrin
0.9	Besnik Murtezan	Aggiornato sezioni 3.2.2, 3.2.4, 4.7.1	02/03/2026	Stefano Maso
0.8	Besnik Murtezan	Aggiornato controllo di flusso (sez. 3.2.2)	15/02/2026	Stefano Maso
0.7	Besnik Murtezan	Stesura sottosezioni (4.3 fino a 4.12)	20/12/2025	Ruben Spadiliero
0.6	Besnik Murtezan	Miglioramento Forma Espositiva e compilazione sottosezioni mancanti (fino a 4.2)	14/12/2025	Ruben Spadiliero
0.5	Stefano Maso	Correzione errori e stesura sottosezioni mancanti	8/12/2025	Gabriele Disa
0.4	Stefano Maso	Continuazione stesura sezione 3, creazione sezione 4 (Processi organizzativi)	27/11/2025	Gabriele Disa
0.3	Stefano Maso	Terminata sezione 2 (Processi primari), inizio sezione 3 (Processi di supporto)	27/11/2025	Gabriele Disa
0.2	Stefano Maso	Stesura fino a sezione 2.4 e correzione prima stesura	25/11/2025	Gabriele Disa
0.1	Stefano Maso	Prima versione	20/11/2025	Gabriele Disa



Indice

1	Introduzione	5
1.1	Scopo del documento	5
1.2	Scopo del prodotto	5
1.3	Riferimenti	5
1.3.1	Riferimenti normativi	5
1.3.2	Riferimenti informativi	5
2	Processi primari	5
2.1	Descrizione	5
2.2	Acquisizione	5
2.2.1	Descrizione	5
2.2.2	Valutazione dei capitolati	5
2.2.3	Aggiudicazione appalto	5
2.3	Fornitura	6
2.3.1	Descrizione	6
2.3.2	Glossario	6
2.3.3	Piano di Progetto	6
2.3.4	Piano di Qualifica	6
2.3.5	Rilascio	6
2.4	Sviluppo	6
2.4.1	Descrizione	6
2.4.2	Analisi dei Requisiti	7
2.4.3	Progettazione	7
2.4.4	Codifica	8
3	Processi di supporto	8
3.1	Documentazione	8
3.1.1	Tipi di documenti	8
3.1.2	Strumenti creazione documento	9
3.1.3	Struttura documenti	9
3.1.4	Significato versioni	9
3.1.5	Ciclo di vita documenti	9
3.2	Controllo di configurazione	10
3.2.1	GitHub e Git	10
3.2.2	Controllo di flusso	10
3.2.3	Scopo e struttura repository "Documentazine-SWE"	11
3.2.4	Scopo e struttura repository "Proof-of-Concept"	11
3.2.5	Scopo e struttura repository "SecondBrain"	12
3.3	Gestione della qualità	12
3.3.1	Descrizione	12
3.3.2	Obiettivi	12
3.4	Verifica	12
3.4.1	Descrizione	12
3.4.2	Obiettivi	12
3.4.3	Analisi statica	13
3.4.4	Analisi dinamica	13
3.4.5	Testing	13
3.5	Validazione	14
3.5.1	Scopo ed aspettative	14



4	Processi organizzativi	14
4.1	Responsabile	14
4.1.1	Gestione Sprint	14
4.1.2	Approvazione Artefatti Baseline	15
4.2	Amministratore	15
4.3	Analista	15
4.4	Progettista	15
4.5	Verificatore	16
4.6	Programmatore	16
4.6.1	Integrazione Codice nella Repository	16
4.7	Gestione di Progetto	16
4.7.1	Allineamento Organizzativo	16
4.8	Organizzazione del Lavoro	17
4.8.1	Modello di Sviluppo	17
4.8.2	Rotazione Ruoli	17
4.8.3	Gestione Attività	17
4.9	Infrastruttura	18
4.10	Etica Lavorativa	18
4.11	Formazione	18



1 Introduzione

1.1 Scopo del documento

Lo scopo del seguente documento è quello di definire il *Way of Working*^G e le *best practices* che ciascun membro del gruppo dovrà impegnarsi a seguire al fine di perseguire obiettivi di qualità ed omogeneità del lavoro e conseguentemente del prodotto. Il documento subirà incrementi con l'avanzare del tempo che richiederanno quindi una visione regolare da parte del gruppo.

1.2 Scopo del prodotto

Il prodotto finale, denominato *Second Brain*, propone di realizzare un software che permetta la scrittura di testo con marcatori in formato Markdown^G, la sua visualizzazione formattata correttamente e l'interazione con un LLM. In particolare, l'utilizzo di LLM^G verrà sfruttato da *Second Brain* per fornire funzionalità^G di riassunto, riscrittura, traduzione e generazione di testo in aggiunta a una forma di critica secondo il modello dei "Sei Cappelli per Pensare" di Edward De Bono.

1.3 Riferimenti

1.3.1 Riferimenti normativi

1. Capitolato^G C6 - Progetto^G Second Brain:
<https://www.math.unipd.it/~tullio/IS-1/2025/Progetto/C6.pdf>
2. Regolamento progetto^G didattico:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/PD1.pdf>

1.3.2 Riferimenti informativi

1. Lezione "*I processi di ciclo di vita del software (T2)*" del corso di *Ingegneria del software*^G:
<https://www.math.unipd.it/~tullio/IS-1/2025/Dispense/T02.pdf>
2. Glossario^G v1.0

2 Processi primari

2.1 Descrizione

Con "processi primari" si intendono "i processi che hanno come clienti soggetti che sono esterni al gruppo".

2.2 Acquisizione

2.2.1 Descrizione

Nel processo detto di acquisizione avviene la raccolta di tutti i requisiti e la loro comprensione, così da poter individuare un capitolato^G adatto al gruppo.

2.2.2 Valutazione dei capitolati

Il gruppo ha esaminato le proposte presentate dai vari proponenti sulla base della loro complessità, delle tecnologie necessarie e dell'effettivo interesse che tali progetti hanno suscitato nei membri del gruppo.

2.2.3 Aggiudicazione appalto

Il gruppo si è candidato per il capitolato^G C6 dell'azienda Zucchetti. Dopo un primo riscontro negativo con il committente, il gruppo è riuscito ad aggiudicarsi l'appalto desiderato.



2.3 Fornitura

2.3.1 Descrizione

Il processo di fornitura consiste nel chiarire tutti i dubbi del proponente^G per quanto riguarda il prodotto finale atteso. Il gruppo si impegna nel comunicare con l'azienda così da determinare i requisiti da dover soddisfare e ricevere riscontri e consigli in fase di sviluppo^G su quanto è stato fatto.

2.3.2 Glossario

Il documento Glossario^G ha l'obiettivo^G di chiarire il significato dei diversi termini utilizzati nell'intera documentazione, in modo da evitare fraintendimenti e ambiguità. È destinato a membri del gruppo, committenti e proponenti.

2.3.3 Piano di Progetto

Il documento *Piano di Progetto*^G contiene la pianificazione temporale per le attività che devono essere svolte in modo da rispettare la data di consegna prefissata. È composto da:

- **Analisi dei Rischi:** indica le difficoltà che il gruppo potrebbe incontrare durante lo svolgimento del progetto^G
- **Modello di sviluppo:** definisce l'insieme dei processi da privilegiare nel ciclo di vita.
- **Pianificazione:** scandisce le attività nel tempo e l'assegnazione delle relative risorse.
- **Preventivo:** definisce un'aspettativa dell'andamento e del termine delle attività.
- **Consuntivo:** mostra l'andamento effettivo del gruppo in retrospettiva al preventivo.

2.3.4 Piano di Qualifica

Il documento Piano di Qualifica^G serve a elencare le attività svolte dal verificatore^G per garantire la qualità del prodotto finale. È composto da:

- **Qualità di processo:** attività da svolgere per monitorare l'andamento dei processi e la loro qualità.
- **Qualità di prodotto:** attività da svolgere per ottenere software di qualità.
- **Verifica:** analisi statica e dinamica effettuata sul prodotto per garantirne il corretto soddisfacimento dei requisiti.
- **Resoconto:** considerazioni effettuate sulla base di risultati della verifica.
- **Validazione:** accertamento finale della completa conformità del prodotto.

2.3.5 Rilascio

Una volta terminato, il prodotto verrà collaudato in presenza di committente e referente. In caso di esito positivo il prodotto verrà consegnato al committente assieme a tutta la documentazione del progetto. Il gruppo non effettuerà nessuna attività ulteriore sul prodotto.

2.4 Sviluppo

2.4.1 Descrizione

L'obiettivo^G del processo di sviluppo^G è quello di far transitare il prodotto attraverso le fasi necessarie per arrivare al rilascio del prodotto in conformità con i requisiti.



2.4.2 Analisi dei Requisiti

L'Analisi dei Requisiti^G è l'attività in cui vengono elencati tutti i requisiti del prodotto finale, i quali verranno poi confermati con committente e referente ed avranno valenza contrattuale. Serve anche come documentazione del prodotto, in quanto contiene tutte le relative funzionalità^G che verranno fornite.

Il documento di Analisi dei Requisiti^G contiene:

- **Attori:** ruoli dell'utente nell'interazione con il sistema.
- **Casi d'uso:** insieme di scenari con uno scopo finale per gli attori, ciascuno con un diagramma a esso associato e la sua descrizione che segue una struttura standard (Attori, Precondizioni, Postcondizioni, Scenario primario e Scenari alternativi).
- **Requisiti:** caratteristiche del prodotto mirate a soddisfare le necessità dell'utente; possono essere:
 - *Funzionali:* si riferiscono alle funzionalità^G che dovranno essere disponibili sul prodotto finale.
 - *Qualitativi:* si riferiscono alle caratteristiche di qualità e performance del prodotto (e.g. Uso Memoria, Tempo, Usabilità, etc).
 - *Vincolanti:* si riferiscono ai limiti imposti sul prodotto che non sono riconducibili a funzionalità^G o qualità (e.g. Tecnologie da utilizzare, vincoli legali, etc...)

In base alla loro priorità si suddividono in:

- *Obbligatorie:* irrinunciabili per l'approvazione del prodotto finale
- *Desiderabili:* presentano buon valore aggiunto ma non strettamente necessari
- *Opzionali:* contrattabili durante lo sviluppo^G del prodotto

Ogni requisito sarà formato da un codice univoco, una descrizione testuale ed un riferimento ad uno o più casi d'uso. I diagrammi UML dei casi d'uso^G devono essere inseriti direttamente nel codice sorgente .tex utilizzando il pacchetto "tikz-uml" seguendo la sintassi prevista.

([Link Documentazione tikz-uml](#))

2.4.3 Progettazione

Va a definire la struttura del progetto^G sulla base dell'Analisi dei Requisiti. In questa fase si prevede la stesura del documento di *Specifiche Tecniche*^G, che va a definire in modo esaustivo:

- Progettazione^G architetturale
- Design^G dell'interfaccia
- Progettazione^G dettagliata

Il documento ha l'obiettivo di eliminare eventuali ambiguità nel successivo processo di codifica, coprendo i requisiti precedentemente analizzati attraverso la definizione dell'architettura del prodotto. Per garantire la qualità del sistema, il documento include la definizione dei processi di testing, che vanno a coprire gli aspetti chiave dell'architettura:

- **Efficienza^G:** capacità del sistema di utilizzare in modo ottimale le risorse disponibili (CPU, memoria, rete), garantendo tempi di risposta adeguati e buone prestazioni anche sotto carico.
- **Modularità:** capacità del sistema di essere suddiviso in componenti indipendenti e riutilizzabili, riducendo le dipendenze tra le parti e facilitando manutenzione ed evoluzione.
- **Robustezza:** capacità del sistema di gestire condizioni anomale o errori di multi-utenza, senza interrompere il funzionamento, mantenendo uno stato consistente.



- **Affidabilità:** capacità del sistema di funzionare correttamente nel tempo, senza guasti, garantendo risultati coerenti e prevedibili.
- **Sicurezza:** capacità del sistema di proteggere dati e risorse da accessi non autorizzati, attacchi e vulnerabilità, garantendo confidenzialità, integrità e disponibilità.
- **Adeguatezza:** capacità del sistema di soddisfare i requisiti funzionali e non funzionali definiti, risultando idoneo allo scopo per cui è stato progettato.
- **Semplicità:** capacità del sistema di essere facilmente comprensibile, utilizzabile e manutenibile, grazie a soluzioni progettuali chiare e non eccessivamente complesse.
- **Portabilità:** capacità del sistema di essere eseguito in ambienti diversi (hardware/software) con modifiche minime o nulle, spesso garantita tramite tecnologie come containerizzazione (Docker^G).

2.4.4 Codifica

In seguito all'analisi e alla progettazione^G, i programmatori si occuperanno d'implementare tutte le funzionalità^G previste per il prodotto finale, basandosi unicamente sui requisiti specificati e sull'architettura definita in precedenza. Sempre in questa fase devono essere implementati i vari test^G per assicurare un corretto funzionamento del prodotto.

Per la stesura del codice si sono adottate delle norme comuni tra cui:

- **Convenzioni di denominazione:** dare nomi significativi a variabili, funzioni, classi, ecc. utilizzando opportunamente lettere maiuscole e minuscole per identificare le parole chiave
- **Indentazione:** usare l'indentazione in modo consistente per evidenziare la struttura del codice
- **Commenti:** aggiungere commenti per spiegare parti di codice complesse, evitando commenti ovvi o ridondanti
- **Lunghezza delle righe:** limitare la lunghezza delle righe per facilitare la lettura e revisione del codice
- **Struttura del codice:** organizzare il codice in blocchi logici e funzioni coese; utilizzare funzioni e classi per evitare la duplicazione del codice
- **Gestione delle eccezioni:** trattare le eccezioni in modo appropriato, usando blocchi try-catch quando necessario
- **Separazione logica di business e logica di interfaccia:** mantenere separata la logica di business dal codice di interfaccia utente o di accesso ai dati, in modo da rendere il codice modulare e facile da testare

Le metriche individuate:

- PD06-FD: Failure density
- PD07-CC: Complessità ciclomatica
- PC12-LOC: Linee di codice

3 Processi di supporto

3.1 Documentazione

3.1.1 Tipi di documenti

È possibile distinguere i documenti in due categorie: quelli ad uso interno e quelli ad uso esterno. Per quanto riguarda quelli ad uso interno abbiamo:



- Verbali interni
- Norme di Progetto^G

Quelli ad uso esterno invece comprendono:

- Verbali esterni
- Analisi dei Requisiti^G
- Piano di Qualifica^G
- Piano di Progetto^G
- Specifica Tecnica^G
- Manuale Utente^G

3.1.2 Strumenti creazione documento

Per la creazione dei documenti inizialmente è stato deciso di utilizzare Overleaf, editor^G LaTeX^G online, ma viste le limitazioni sulla stesura collaborativa (limite di due persone per documento), è stato scelto di utilizzare Visual Studio Code con l'estensione LaTeX^G Workshop per poter lavorare in locale. Ciascun membro del gruppo, in seguito ad una modifica fatta al documento in locale, si impegna ad aggiornare anche il documento presente sulla repository^G, in modo da rendere sempre disponibile l'ultima versione del documento al resto del gruppo. ([Link Documentazione LaTeX^G](#))

3.1.3 Struttura documenti

Ciascun documento ha una struttura ben definita:

- *Prima pagina*: contiene nome e logo del gruppo, informazioni del documento e la data in cui è stato approvato.
- *Tabella delle revisioni*: una tabella che contiene informazioni del versionamento del documento: la versione, l'autore, descrizione delle modifiche apportate, la data, il verificatore^G delle modifiche.
- *Indice*: elenco dei contenuti del documento.
- *Sezioni*: contenuti effettivi.

3.1.4 Significato versioni

Ogni documento presente nella repository^G è definito dalla sua versione corrente $x.y$ (visibile nella tabella delle revisioni), con il seguente significato:

- x : è un numero intero, parte da zero e viene incrementato ogni volta che il documento raggiunge lo stato finale per una baseline. (es: RTB^G $\rightarrow$ 1.y ; PB $\rightarrow$ 2.y)
- y : è un numero intero, parte da 1 e rappresenta lo stato di verifica^G di un documento.

3.1.5 Ciclo di vita documenti

Un documento attraversa le seguenti fasi:

- *Pianificazione*: il responsabile^G pianifica la creazione di un documento nuovo e la assegna ad uno o più redattori.
- *Stesura iniziale*: i redattori incaricati si occupano di creare il documento nella sua versione iniziale 0.1.
- *Sviluppo*^G: il documento attraversa un ciclo di iterazioni composte da:



- *Aggiornamento*: i contenuti del documento vengono aggiunti, corretti o eliminati, incrementando la versione $x.y \rightarrow x.y+1$.
- *Verifica^G*: il documento viene verificato da un membro del gruppo diverso da quelli che hanno lavorato sull'aggiornamento. In caso di successo, la versione del documento presente nella repository^G viene aggiornata con la versione dell'iterazione di sviluppo.
- *Approvazione*: il documento raggiunge lo stato di maturazione sufficiente per la prossima baseline^G, aggiornando la versione a $x.0$.

3.2 Controllo di configurazione

3.2.1 GitHub e Git

Il gruppo ha deciso di utilizzare Git^G come strumento di versionamento locale e remoto e GitHub^G come servizio cloud in cui mantenere la repository^G completa sempre accessibile da tutti i membri del gruppo. ([Link Sito Ufficiale Git^G](#)) ([Link GitHub^G](#))

3.2.2 Controllo di flusso

Ogni membro dovrà adottare il seguente flusso di lavoro quando si prende in carico una issue^G dalla sezione **Ready**, sia per documentazione che per codice sorgente:

1. Spostare la issue^G presa in carico da **Ready** a **In progress** su GitHub.
2. Assicurarsi di avere la repository^G sulla propria macchina locale. In caso contrario clonare la repository^G remota. ([Guida Clonazione GitHub^G](#))
3. Aggiornare la versione locale della repository^G con l'ultima versione disponibile del ramo remoto **main** (*comando* `gitG pull`).
4. Creare un branch^G locale con il nome della feature/documento a cui si sta lavorando (meglio se riconducibile alla issue^G presa in carico). (*comando* `gitG branchG`)
5. Eseguire il lavoro solamente sul branch^G locale.
6. Tracciare il lavoro del branch^G locale sulla repository^G remota (*comando* `gitG push -u origin <nome-branch-locale>`) così da attivare il meccanismo automatico di Pull-Request verso il ramo **main** remoto.
7. Spostare la issue^G da **In progress** a **In review** su GitHub.
8. Attendere la verifica^G della PR da parte di un verificatore. È opportuno non effettuare altre modifiche al branch^G che sta attendendo la PR. In base al risultato della verifica^G:
 - Esito Positivo
 - (a) Chiudere il proprio branch^G locale (utile per evitare confusione con branch^G di lavoro successivi).
 - Esito Negativo
 - (a) Ritornare al passo 5, correggendo il lavoro in base alle indicazioni del verificatore.

Per i verificatori il flusso di verifica^G di una determinata issue^G della sezione **In review** è il seguente:

1. Aprire su GitHub^G la Pull Request relativa alla issue.
2. Eseguire la review dei file aggiunti e modificati dalla PR.
 - In caso di esito positivo:
3. **Solo per documentazione**: inserire il proprio nome nella cella di verifica^G della versione corrente (tabella delle revisioni).



4. Approvare il merge della Pull Request.
 5. Eliminare il branch^G remoto da cui è partita la PR.
 6. Spostare la issue^G da **In review** a **Done**.
- In caso di esito negativo:
3. Restituire un feedback all'autore con le modifiche da eseguire per correggere gli errori rilevati, **senza chiudere ne approvare la PR**.
 4. Spostare la issue^G da **In review** a **In progress**.

Per quanto riguarda il Glossario^G, l'aggiunta di un termine viene svolta nella seguente maniera:

1. Creare un nuovo branch^G
2. Aprire il file *glossario.json*.
3. Aggiungere un nuovo dizionario termine-definizione all'interno dell'array relativo all'iniziale del termine.
4. Pubblicare il branch^G (in automatico viene generata una Pull Request)

La creazione delle sezioni del documento Glossario^G e l'inserimento delle *G* nei documenti avviene automaticamente tramite GitHub^G Action (nel caso in cui la Pull Request venga approvata).

3.2.3 Scopo e struttura repository "Documentazine-SWE"

La repository^G ha la funzione di mantenere una versione aggiornata dei file che dovranno produrre documentazione, ovvero i file *.tex*.

La struttura della repository^G è la seguente:

- Candidatura^G
- RTB^G
- Verbali
 - Interni
 - Esterni

3.2.4 Scopo e struttura repository "Proof-of-Concept"

La repository^G ha lo scopo di mantenere il versionamento dei file relativi al Proof-of-Concept. La struttura è la seguente:

- *docker/nginx* : cartella contenente file di configurazione per integrazione nginx-docker.
- *src* : cartella del codice sorgente (molti file sono ausiliari al framework^G Laravel).
- *.dockerignore* : file ausiliario per docker.
- *Dockerfile* : file di configurazione per generare immagine Docker^G della business logic.
- *README.md* : file di istruzioni per il corretto avvio dell'applicazione.
- *docker-compose.yml* : file di configurazione per generare i container^G dell'applicazione.



3.2.5 Scopo e struttura repository "SecondBrain"

Le repository^G ha la funzione di mantenere il codice sorgente del progetto^G del capitolato^G C6. La struttura è la seguente:

- `docker/nginx` : cartella contenente file di configurazione per integrazione nginx-docker.
- `docker/php/Dockerfile` : file di configurazione per generare immagine Docker^G della business logic.
- `src` : cartella del codice sorgente (molti file sono ausiliari al framework^G Laravel).
- `Makefile` : file di automazione che definisce comandi di build, avvio, test^G e arresto per la manutenzione del progetto^G tramite Docker^G Compose.
- `README.md` : file di istruzione per la corretta manutenzione dell'applicazione.
- `docker-compose.yml` : file di configurazione per generare i container^G dell'applicazione.
- `docker-compose.dev.yml` : file di configurazione per generare i container^G dell'applicazione per l'ambiente di development.
- `docker-compose.prod.yml` : file di configurazione per generare i container^G dell'applicazione per l'ambiente di produzione.

3.3 Gestione della qualità

3.3.1 Descrizione

La gestione della qualità è un insieme di processi che hanno lo scopo di garantire che software, documenti ed ulteriori artefatti, prodotti durante i processi di ciclo di vita del progetto^G, aderiscano a degli standard di qualità rispetto a requisiti specifici, in modo da soddisfare il più possibile le aspettative di utente finale e proponente.

3.3.2 Obiettivi

La gestione della qualità mira a raggiungere i seguenti obiettivi:

- Realizzare un prodotto di qualità, in linea con quanto richiesto dal proponente.
- Ridurre al minimo rischi che potrebbero danneggiare la qualità del prodotto.
- Rispettare il budget preventivato.

Le modalità e gli strumenti utilizzati per la gestione della qualità dei processi e del prodotto sono definiti nel documento "*Piano di Qualifica*".

3.4 Verifica

3.4.1 Descrizione

La verifica^G è un processo che valuta il prodotto durante tutte le fasi, dalla progettazione^G fino al rilascio. Il suo obiettivo^G è quello di garantire che il software rispetti aspettative e requisiti.

3.4.2 Obiettivi

La verifica^G si propone di raggiungere i seguenti obiettivi:

- Individuare errori e anomalie il prima possibile per limitarne i costi derivati ("*Fail Fast*")
- Assicurarsi che il prodotto non regredisca su errori precedentemente corretti
- Monitorare ed assicurare la qualità del prodotto in ogni momento

Nel documento "*Piano di Qualifica*" vengono definiti gli aspetti da verificare, i criteri di accettazione da impiegare e gli obiettivi da raggiungere.



3.4.3 Analisi statica

L'analisi statica è una metodologia di verifica^G che non richiede l'esecuzione del prodotto e si basa su revisione del codice e della documentazione. Lo scopo è quello di verificare l'assenza di difetti e la conformità ai requisiti.

Viene messa in atto attraverso due tecniche di lettura:

- **Walkthrough:** tecnica che coinvolge autore dell'oggetto di verifica^G e verificatore^G nella revisione sequenziale del codice e della documentazione nella loro interezza, con discussione dei problemi incontrati;
- **Inspection:** tecnica che consiste nel revisionare parti specifiche del codice e della documentazione seguendo checklist nel momento in cui si ha un'idea di dove possano esserci problemi;

È preferibile utilizzare la tecnica *Inspection* in quanto più efficiente ricordando che non sarà adottabile sin dall'inizio in quanto necessita di esperienza per poter costruire delle checklist corrette.

3.4.4 Analisi dinamica

L'analisi dinamica è una metodologia di verifica^G che si basa sull'esecuzione del codice con l'intento di trovare difetti. In questa fase vengono svolti i test^G per individuare e verificare il comportamento del software, puntando il più possibile all'automazione in quanto test^G manuali sono altamente onerosi. I dettagli relativi a questa fase sono presenti nel *Piano di Qualifica*^G.

3.4.5 Testing

L'obiettivo^G del testing è garantire il corretto funzionamento delle componenti, verificando che producano i risultati attesi. Questi test^G sono responsabili di individuare eventuali anomalie di funzionamento.

Test di unità

I test^G di unità costituiscono un pilastro nella verifica^G della correttezza delle singole unità di codice all'interno di un sistema software. Essi sono progettati per isolare e verificare singoli componenti, come funzioni o metodi, garantendo che ciascuno di essi funzioni correttamente e produca risultati attesi in conformità alle specifiche.

L'obiettivo^G primario dei test^G di unità è garantire l'integrità e la correttezza di ogni singola unità di codice prima della sua integrazione con il resto del sistema. Questo approccio consente di individuare e correggere eventuali errori o bug nelle fasi iniziali dello sviluppo^G, riducendo così il rischio di problemi più gravi nel sistema finale.

È possibile utilizzare oggetti simulati o parziali al fine di isolare l'unità di codice in esame dalle sue dipendenze esterne. Questo permette di verificare il comportamento dell'unità in contesti controllati e di garantire un'efficace separazione durante le fasi di test. Inoltre, i test^G di unità sono progettati per raggiungere una copertura completa dei percorsi all'interno dell'unità di codice. Ciò significa che vengono creati appositamente test^G per attivare e verificare specifici percorsi nel codice, al fine di garantire la copertura completa e accurata delle varie parti dell'unità.

Test di sistema

L'obiettivo^G dei test^G di sistema è garantire che il sistema soddisfi i requisiti funzionali e non funzionali specificati durante la fase di progettazione^G e sviluppo. Essi mirano a convalidare le funzionalità^G del sistema nel contesto delle attività e dei processi previsti, verificando che il sistema si comporti correttamente e risponda in modo appropriato a tutte le richieste e interazioni degli utenti.

Vengono simulati scenari realistici e casi d'uso^G tipici che possono verificarsi durante l'utilizzo quotidiano del sistema.

Test di accettazione L'obiettivo^G principale dei test^G di accettazione è convalidare che il software sia in grado di risolvere efficacemente i problemi e soddisfare le esigenze degli utenti.

Essi valutano il sistema rispetto a casi d'uso^G reali e alle specifiche funzionali concordate, verificando se il software si comporta come previsto e se fornisce i risultati desiderati. Tali test^G avverranno in presenza dell'azienda proponente.



Formato dei test

La classificazione dei test^G avviene seguendo il seguente formato:

T[Tipo]-Codice

dove il **Tipo** può essere "U" per i test^G di unità e "S" per i test^G di sistema, oppure "A" per quelli di accettazione.

3.5 Validazione

3.5.1 Scopo ed aspettative

La validazione^G è il processo che verifica^G se un prodotto software è conforme ai requisiti e alle aspettative del cliente. È un processo interno svolto una volta sola alla fine dello sviluppo^G, precedendo e abilitando il collaudo del prodotto con il committente in caso di successo dei test^G di validazione^G (definiti sempre nel "*Piano di Qualifica*"). Gli aspetti principali a cui il prodotto deve far fronte sono:

- **Conformità ai requisiti:** soddisfare tutti i requisiti specificati dal cliente nell'Analisi Requisiti
- **Funzionamento corretto:** funzionare senza presentare errori durante l'utilizzo
- **Efficacia:** offrire funzionalità^G che effettivamente soddisfano i bisogni dell'utente
- **Usabilità:** essere il più possibile facile, chiaro ed intuitivo da utilizzare:

4 Processi organizzativi

Questa sezione è dedicata a fornire una breve descrizione di ruoli e responsabilità dei membri del gruppo.

4.1 Responsabile

Il responsabile^G è la figura che guida e coordina le attività per la realizzazione del progetto. Le sue responsabilità includono:

- Pianificare e coordinare le attività di progetto^G
- Assegnare le attività ai membri del gruppo sulla base dei ruoli di ciascuno
- Gestire lo Sprint^G e le milestone^G
- Monitorare lo stato e l'andamento delle attività
- Mantenere aggiornato il "*Piano di Progetto*"

4.1.1 Gestione Sprint

All'inizio di ogni sprint^G il responsabile^G dovrà:

1. Determinare la durata dello sprint^G
2. Individuare le attività che si dovranno svolgere durante lo sprint^G ("*Sprint^G Backlog*")
3. Creare le Issues tramite GitHub^G, associarle alle attività ed assegnarle ai relativi membri del gruppo
4. Aggiornare il documento di *Piano di Progetto*^G con le informazioni relative al nuovo sprint^G



4.1.2 Approvazione Artefatti Baseline

Il responsabile^G deve valutare quando un documento, o una determinata componente codificata, abbia raggiunto uno stato sufficiente per la baseline^G pianificata ed approvarne la versione finale per tale baseline.

4.2 Amministratore

Il ruolo di amministratore^G si occupa di definire, mantenere e controllare l'ambiente IT di lavoro del gruppo, in maniera più proattiva possibile. In particolare si focalizza su:

- Selezionare ed attivare le risorse informatiche a supporto del *Way of Working*^G
- Risolvere problemi riguardanti l'ambiente di lavoro su segnalazione degli altri membri
- Ricercare risorse per migliorare continuamente l'ambiente di lavoro
- Automatizzare il più possibile il funzionamento dell'ambiente di lavoro
- Monitorare lo stato del controllo di versione e configurazione
- Attuare piani e procedure per la gestione della qualità

4.3 Analista

Ruolo essenziale per il successo del progetto^G; si occupa di analizzare le specifiche del proponente^G, presenti nel capitolato^G, ed il dominio di interesse del progetto^G al fine di individuare tutti i requisiti che il prodotto finale dovrà soddisfare per essere conforme alle richieste del proponente. Le responsabilità dell'analista^G riguardano:

- Analizzare il problema ed il relativo dominio di utilizzo
- Redigere il documento "*Analisi dei Requisiti*"
- Studiare i casi d'uso^G e realizzare i relativi diagrammi UML
- Aggiornare i documenti Piano di progetto^G e Piano di Qualifica^G

Gli analisti non seguono il progetto^G fino alla consegna.

4.4 Progettista

Il progettista^G si occupa di determinare le scelte realizzative sulla base dei requisiti individuati dagli analisti. Ha competenze tecniche e tecnologiche aggiornate. Si impegna a fornire una struttura robusta e semplice per la codifica mantendendo un costo organizzativo ridotto rispetto al beneficio della progettazione^G in dettaglio. In particolare si occuperà di:

- Individuare e definire una Architettura generale (progettazione^G logica)
- Suddividere l'architettura in profondità puntando a modularità e unità il più possibile atomiche (progettazione^G di dettaglio)
- Garantire Efficienza^G ed Efficacia^G dell'architettura
- Ricercare semplicità ed aumentare la complessità solo se necessario
- Definire la specifica tecnica^G



4.5 Verificatore

Il verificatore^G si occupa di monitorare la correttezza del lavoro svolto dagli altri componenti sulla base delle proprie competenze tecniche e della norme di progetto. Le sue priorità sono:

- Valutare le Pull Request utilizzando gli strumenti di verifica^G individuati nel *Piano di Qualifica*^G
- Verificare i documenti prodotti dai relativi redattori
- Verificare il codice prodotto dai programmatori
- Monitorare ed occuparsi delle Issue^G nella sezione **In review**

4.6 Programmatore

Il programmatore^G si occupa di codificare tutte le componenti individuate dalla progettazione^G insieme agli elementi ausiliari di verifica^G e validazione. Il suo lavoro si focalizza su:

- Implementare la specifica tecnica^G realizzata dal progettista^G in maniera rigorosa
- Implementare gli elementi necessari alla verifica^G e la loro eventuale integrazione con strumenti terzi
- Realizzare la documentazione relativa al codice prodotto
- Seguire espressamente le norme di progetto^G per poter produrre codice di qualità

4.6.1 Integrazione Codice nella Repository

Per ogni funzionalità^G che richiede la codifica, il programmatore^G deve creare un branch^G di sviluppo^G con il nome della funzionalità^G interessata. Ogni contribuzione di codice va effettuata su tale branch^G e quando il programmatore^G ritiene di aver completato la codifica per quella funzionalità^G deve procedere manualmente ad aprire una Pull Request di merge sul main, la quale deve poi essere valutata dal verificatore^G e, in caso di successo, si conclude con l'integrazione del codice sul ramo main della repository.

4.7 Gestione di Progetto

4.7.1 Allineamento Organizzativo

Vengono utilizzate due modalità di comunicazione per allinearsi e coordinarsi sulle attività da svolgere:

- **Comunicazioni Interne:** coinvolgono tutti i membri del gruppo e avvengono attraverso i seguenti strumenti:
 - *WhatsApp*: utilizzato per comunicazioni veloci, segnalazioni, dubbi e organizzazione di incontri formali in forma scritta.
Sito Ufficiale WhatsApp
 - *Discord*: utilizzato per incontri interni formali, documentati con un verbale, al fine di discutere delle questioni principali relative al prodotto. Si comunica in forma vocale con eventuali condivisioni di schermo per avere visione comune sull'argomento in questione.
Sito Ufficiale Discord
- **Comunicazioni Esterne:** coinvolgono gruppo, proponente^G e/o committente; si usufruisce dei seguenti strumenti:
 - *Posta Elettronica*: utilizzata per comunicare con proponente^G o committente a fini organizzativi o per chiarimenti.
E-Mail del gruppo: tool8eight@gmail.com



- *Google Meet*: utilizzato per riunioni esterne con il proponente^G in forma di videochiamata.
Sito Google Meet

Per la codifica del prodotto viene richiesto l'utilizzo di:

- **Laravel** : framework^G PHP per lo sviluppo^G del prodotto (sia per PoC che MVP^G); è necessario seguire il più possibile le metodologie proprie del framework.
Sito Ufficiale Laravel
- **Docker^G Desktop** : strumento che permette l'utilizzo di container^G al fine di avere un ambiente di sviluppo^G uguale per tutti.
Sito Ufficiale Docker^G

4.8 Organizzazione del Lavoro

4.8.1 Modello di Sviluppo

Il gruppo ha deciso di adottare un modello basato su *SCRUM* dove il lavoro è suddiviso in periodi di incremento di due settimane chiamati Sprint. Tale modello permette di avere un riscontro frequente sullo stato del prodotto finale così da poter capire se la direzione di sviluppo^G è corretta o se sono necessari interventi correttivi. Le fasi di uno sprint^G sono:

- **Sprint Planning**: vengono decisi i ruoli per ogni membro del gruppo e lo *Sprint^G Backlog*, ovvero le attività di lavoro da svolgere nello sprint^G corrente individuate a partire dal *Backlog* intero del progetto. Ogni attività viene poi assegnata ad uno o più membri. In ultimo luogo si fissa la data di fine sprint^G, per una durata totale di circa due settimane. Tale fase viene svolta tramite una riunione interna su Discord con la partecipazione di tutti i membri del gruppo e la stesura di un verbale.
- **Sprint Review**: viene presentato ed analizzato tutto ciò che è stato prodotto al termine dello sprint^G in presenza di tutto il gruppo.
- **Sprint Retrospective**: viene valutato il successo dello sprint^G, analizzando gli aspetti che hanno funzionato maggiormente, gli aspetti che si possono migliorare, novità da introdurre nei prossimi sprint^G ed eventuali caratteristiche da rimuovere. L'intero gruppo partecipa a questa fase.

Link Guida Framework^G SCRUM

4.8.2 Rotazione Ruoli

Ad ogni sprint^G verranno ruotati i ruoli dei membri del gruppo in maniera equa e ragionevole (in base alle disponibilità e necessità di ogni membro) sempre garantendo assenza di conflitti sul lavoro eseguito (e.g. un verificatore^G non può verificare il codice che aveva scritto quando ricopriva il ruolo di programmatore^G). Si ricorda in ogni caso che nessun membro può ricoprire più di un ruolo allo stesso tempo.

4.8.3 Gestione Attività

Per gestire e tener traccia delle attività da svolgere viene utilizzata la funzionalità^G *Projects* all'interno di GitHub. Le attività sono definite tramite *Issue^G* e contengono vari campi che ne indicano le caratteristiche. Ogni *Issue^G* può trovarsi in uno dei seguenti stati:

- **Backlog**: l'attività è stata individuata e fa quindi parte del progetto^G
- **Ready**: l'attività è stata inserita all'interno dello *Sprint^G Backlog* ed è pronta per essere presa in carico dal relativo assegnatario
- **In Progress**: l'assegnatario ha preso in carico l'attività e ci sta lavorando attivamente; vengono compilati i campi della *issue^G* relativi al progresso



- **In Review:** l'attività è stata eseguita dall'assegnatario e attende la verifica^G
- **Done:** l'attività è stata definitivamente completata

Il responsabile^G si occupa delle transizioni *Backlog* -> *Ready* e *In Review* -> *Done* mentre le altre transizioni sono di responsabilità dei relativi assegnatari. In seguito all'effettivo cambiamento avvenuto sull'attività legata alla issue^G è opportuno modificare il relativo stato nella sezione *Project*.

4.9 Infrastruttura

L'intera infrastruttura al momento è basata su GitHub^G senza l'utilizzo di strumenti esterni che accedono alla repository^G, tuttavia potranno avvenire modifiche future con l'avanzamento del progetto.

4.10 Etica Lavorativa

Il gruppo si impegna a seguire il principio di *miglioramento continuo* per tutti gli aspetti del progetto^G al fine di ottenere risultati con qualità incrementale per l'intera durata delle attività. Ogni membro si impegna a svolgere il lavoro al meglio delle proprie capacità e disponibilità con la massima trasparenza e collaborazione.

4.11 Formazione

Ogni membro del gruppo è tenuto ad apprendere in maniera autonoma il funzionamento e l'utilizzo delle varie tecnologie e strumentazioni che vengono utilizzati per lo svolgimento del progetto. Qualora si riscontrassero difficoltà è opportuno avvisare il gruppo al fine di poter ricevere supporto e allineare il più possibile il gruppo sullo stesso livello.